

Diseño y Desarrollo de una Arquitectura Multi-Agente Basada en LangGraph para el Procesamiento y Modelado de Datos

1. Introducción al Paradigma Multi-Agente en Procesamiento de Datos

La evolución contemporánea de los modelos de lenguaje grande (LLMs) ha catalizado una transición paradigmática en el campo de la inteligencia artificial, desplazando el enfoque desde la generación de texto conversacional hacia la orquestación de sistemas autónomos complejos. En el contexto de un Trabajo de Fin de Máster (TFM) orientado a la ciencia de datos y la ingeniería de aprendizaje automático, la implementación de una arquitectura subyacente requiere un diseño meticuloso que garantice determinismo, reproducibilidad computacional y escalabilidad horizontal.¹ El presente documento expone el diseño exhaustivo de una arquitectura basada en el marco LangGraph, estructurada bajo un patrón jerárquico de supervisión, donde el sistema emula una cadena de montaje industrial. En esta analogía funcional, los datos en crudo constituyen la materia prima, mientras que los agentes artificiales actúan como operarios especializados, todos ellos coordinados por una entidad supervisora central.³

El diseño inicial del proyecto demanda una expansión arquitectónica profunda para resolver las discrepancias inherentes a los sistemas generativos puros. Se introducen mejoras críticas, tales como la adopción de tuberías algorítmicas (pipelines) híbridas en lugar de la generación de código fuente desde cero, una gestión de estado persistente e inmutable, y la integración de mecanismos de validación humana (*Human-in-the-loop*).¹ Además, este informe establece una hoja de ruta tecnológica expansiva que detalla las fases de desarrollo futuro. Dicha hoja de ruta abarca desde la ingeniería de *prompts* estructurados para modelos como OpenAI Codex o GPT-4o, pasando por la ejecución distribuida en clústeres de supercomputación mediante el planificador SLURM, hasta culminar en un plan de migración hacia un entorno de producción SaaS (*Software as a Service*). Esta fase de producción contempla la contenerización integral mediante Docker y el despliegue de interfaces de usuario interactivas desarrolladas en FastAPI, React y Flutter.⁴

2. Fundamentos de la Arquitectura de Orquestación con LangGraph

La arquitectura propuesta abandona las aproximaciones de agentes monolíticos en favor de un patrón de supervisor jerárquico. En este paradigma topológico, los agentes especializados carecen de comunicación directa entre sí; en su lugar, son coordinados por un nodo central

que monopoliza el control del flujo de comunicación y asume la delegación de tareas basándose en el análisis continuo del contexto actual.¹ LangGraph, concebido como un marco de orquestación de bajo nivel enfocado estrictamente en flujos de trabajo con estado, permite modelar estas complejas interacciones como un grafo dirigido, el cual puede contener ciclos para habilitar procesos iterativos de corrección y auditoría.²

2.1. El Estado Compartido y la Inmutabilidad de la Información

El pilar estructural de cualquier arquitectura implementada en LangGraph es el Estado (*State*). Antes de proceder con la instanciación de cualquier agente, el grafo exige la definición de un esquema de datos semánticamente estricto. Este estado actúa como una "pizarra global compartida" que muta dinámicamente conforme transita de nodo en nodo.³ El diseño del esquema, típicamente implementado a través de anotaciones de tipado estático como TypedDict o modelos de validación Pydantic en Python, previene inconsistencias de memoria.⁸

Para este TFM, el estado global ha sido diseñado para consolidar múltiples dimensiones de información, evitando que los agentes incurran en un consumo excesivo de memoria o en la lectura redundante de datos masivos. La estructura del estado contempla los siguientes componentes fundamentales:

Componente del Estado	Tipo de Estructura	Descripción y Funcionalidad Teórica
Rutas de Almacenamiento	Cadenas de texto (Strings)	Variables direccionales como <code>raw_path</code> , <code>clean_path</code> y <code>tensor_path</code> . Previenen la sobrecarga de memoria al evitar que los agentes manipulen marcos de datos masivos directamente. ³
Contexto e Instrucciones	Texto estructurado	Directrices globales inyectadas al inicio. Por ejemplo, "Buscamos alta sensibilidad para detectar anomalías tempranas; es crítico evitar falsos positivos". Indispensable para la auditoría. ³
Historial de Mensajes	Lista de objetos de mensaje (MessagesState)	Registro acumulativo de interacciones que mantiene

		el linaje de la conversación, fundamental para que el supervisor comprenda las acciones previas. ⁹
Configuraciones Técnicas	Diccionarios (JSON / Dict)	Parámetros generados por los nodos (tamaños de ventana, métodos de escalado, métricas). Actúan como vectores de configuración para los scripts ejecutores. ³

2.2. Persistencia del Estado y Control Asíncrono

Para asegurar que el proyecto no dependa de ejecuciones volátiles en memoria RAM, la arquitectura integra una capa de persistencia mediante bases de datos relacionales, utilizando bibliotecas como langgraph-checkpoint-postgres.¹¹ Esta persistencia sigue un riguroso ciclo de lectura-ejecución-escritura. Al iniciarse un proceso, el usuario o el disparador programático envía un identificador de hilo (`thread_id`).¹³ LangGraph consulta en la base de datos PostgreSQL el último estado serializado coincidente con dicho identificador y lo carga en memoria.¹³

Conforme el agente procesa la entrada y el grafo transita por sus nodos, el nuevo estado no sobrescribe al anterior; por el contrario, se serializa y se guarda como un nuevo registro o *checkpoint*.¹³ Esta metodología garantiza la inmutabilidad y permite un historial de retroceso (*time-travel debugging*).¹² Además, constituye el núcleo tecnológico que posibilita la pausa de la ejecución computacional ("Dormant State"), liberando recursos mientras el sistema espera la intervención y validación de un operador humano antes de ejecutar tareas intensivas.¹³ Para controlar el crecimiento exponencial de las tablas de PostgreSQL, se implementan políticas de purga que conservan únicamente los *checkpoints* de una ventana temporal reciente, tratando estos registros como registros operacionales y no como almacenamiento a largo plazo.¹⁴

3. Especificación Detallada de Nodos y Transiciones de Estado

El sistema se segrega metodológicamente en seis nodos operativos principales. Esta topología separa la lógica de razonamiento estocástico, delegada a los Modelos de Lenguaje Grande, de la lógica de ejecución matemática y de transformación de datos, la cual recae exclusivamente en código Python determinista. Esta dicotomía arquitectónica soluciona una de las mayores vulnerabilidades de los sistemas de IA: permitir que un agente escriba y ejecute código arbitrario sin restricciones, lo que habitualmente deriva en errores de sintaxis y corrupción de

datos.³

3.1. Nodo 1: El Supervisor Orquestador (El Director)

El Supervisor es la inteligencia central del sistema. Su naturaleza es puramente estratégica; no toca los datos tabulares ni ejecuta manipulaciones vectoriales.³ Su función cardinal consiste en leer el `MessagesState`, evaluar en qué etapa del ciclo de vida del dato se encuentra el proceso, y enrutar la ejecución hacia el nodo especialista correspondiente.¹ Ante un fallo de validación reportado por un agente subordinado, el Supervisor ejerce su capacidad de razonamiento para interpretar la traza del error y decidir, de forma autónoma, si reintenta la operación con instrucciones modificadas o si aborta el conducto completo.³ Adicionalmente, este nodo alberga la interfaz fundamental del sistema *Human-in-the-loop*, deteniendo la propagación del grafo para solicitar aprobación explícita del investigador antes de despachar trabajos hacia infraestructuras de alto rendimiento.²

3.2. Nodo 2: Agente Limpiador (Analista de Calidad)

Dedicado al saneamiento de anomalías, este agente no recibe el conjunto de datos completo, una restricción diseñada para eludir alucinaciones generativas y optimizar el gasto de tokens. En su defecto, se le suministra un diccionario sintético que perfila la tabla: incluye metadatos como la tipología de las columnas, distribuciones estadísticas básicas, el porcentaje de valores faltantes y una muestra representativa de tres filas.³ El agente evalúa este resumen estadístico y emite un veredicto estructurado en un objeto JSON, el cual funciona como un *checklist* prescriptivo de tareas de limpieza.³

Para materializar esta prescripción, el nodo incorpora un **Ejecutor Determinista**. Este componente es un script Python inmutable que procesa sintácticamente el JSON devuelto. Si el documento JSON indica, por ejemplo, que una variable específica requiere la imputación de nulos mediante la mediana, el script aplica directamente la función pertinente sobre el marco de datos (ej. `df.fillna()`) y sobrescribe el archivo en la ruta de almacenamiento limpio.³

3.3. Nodo 3: Agente Estructurador (Ingeniero de Datos)

La transición de datos tabulares a estructuras matemáticas aptas para el aprendizaje profundo requiere decisiones topológicas complejas. El Agente Estructurador sigue un riguroso árbol de decisión estricto para configurar estas transformaciones.³ Define las metodologías de escalado (transformaciones MinMax, estandarización Z-score), configura el particionamiento de los subconjuntos de entrenamiento, validación y prueba, y, en caso de series temporales, establece las dimensiones de las ventanas deslizantes (*Sliding Windows*).³ El resultado es un esquema de configuración adicional que se inyecta en el estado global.³ Paralelamente, su propio ejecutor determinista carga los datos limpios, aplica el preprocesamiento algebraico indicado y exporta los resultados físicos como tensores multidimensionales listos para ser consumidos por arquitecturas neuronales.³

3.4. Nodo 4: Módulo de Modelado Híbrido (El Científico de Datos)

La delegación completa del modelado predictivo a la generación de texto de un agente de inteligencia artificial es inherentemente inestable.³ Por tanto, la arquitectura redefine este componente mediante un modelo de **Pipeline Híbrido**.³ El agente asume el rol de selector estratégico en lugar de programador puro. Basándose en los perfiles de datos estructurados, elige entre una batería de algoritmos pre-empaquetados desarrollados con anterioridad por el usuario humano (por ejemplo, arquitecturas basadas en *Isolation Forest* para detección espacial o *Autoencoders* para reducción de dimensionalidad no lineal).³ El agente determina los hiperparámetros óptimos, pero la ejecución técnica recae en un conducto de Python seguro que efectúa el entrenamiento de manera aislada, capturando las predicciones y consolidando las métricas resultantes sin exposición a errores sintácticos generados dinámicamente.³

3.5. Nodo 5: Evaluador de Métricas (El Auditor)

El proceso de validación del modelo se confía al Evaluador de Métricas. Su responsabilidad es cruzar los resultados empíricos obtenidos por el ejecutor del modelo (tasas de error, Puntuación F1, precisión general) con el contexto heurístico que el Supervisor arrastra en el estado desde la instanciación del proceso.³ Por ejemplo, si el contexto global decreta que la minimización de falsos positivos es crítica para la viabilidad comercial, un modelo con alta sensibilidad pero precisión deficiente será penalizado severamente.³ Tras la deliberación, emite un juicio: aprueba el modelo o formula retroalimentación constructiva para el Supervisor, sugiriendo ajustes iterativos como la modificación de las ventanas temporales en el nodo Estructurador.³

3.6. Nodo 6: Agente Redactor (El Comunicador)

El nodo terminal del grafo recopila la totalidad del estado persistido a lo largo del proceso. Condensa las acciones de limpieza documentadas, la configuración de la matriz de datos, la fundamentación teórica de la elección del algoritmo y la auditoría final de las métricas de rendimiento.³ Con esta ingente cantidad de información, redacta un documento exhaustivo en formato Markdown o PDF. Este informe garantiza la transparencia algorítmica y sirve como un documento de auditoría formal para respaldar la integridad del experimento.³

4. Dinámica y Representación Visual del Flujo de Trabajo

La intrincada coreografía de dependencias, evaluaciones condicionales y ejecuciones deterministas demanda una representación topológica formal. Utilizando la sintaxis de grafos de Mermaid, un estándar adoptado de manera nativa por ecosistemas como LangGraph para la trazabilidad visual¹⁶, se delinea el ciclo de vida de la ejecución del sistema multi-agente.¹⁸

Fragmento de código

```
graph TD
  %% Nodos Iniciales y de Estado
  START((INICIO))
  State_Init[Iniciación del Estado Compartido]
  Supervisor{Supervisor\nEnrutador Central}

  %% Nodos de Limpieza
  Limpiador_Agent
  Limpiador_Exec

  %% Nodos de Ingeniería
  Estructurador_Agent[Agente Estructurador\nConfiguración Hiperparamétrica]
  Estructurador_Exec

  %% Intervención Humana
  Human_Review{Punto de Control\nHuman-in-the-Loop}

  %% Nodos de Modelado
  Modelador_Agent
  Modelador_Exec[Ejecutor Híbrido\nEntrenamiento y Predicción]

  %% Nodos de Auditoría y Reporte
  Evaluador
  Redactor

  END((FIN))

  %% Relaciones de Flujo
  START --> State_Init
  State_Init --> Supervisor

  %% Bucle Operativo 1: Limpieza
  Supervisor -- "Delegación Fase 1" --> Limpiador_Agent
  Limpiador_Agent -- "Generación JSON Estricto" --> Limpiador_Exec
  Limpiador_Exec -- "Actualización de Estado" --> Supervisor

  %% Bucle Operativo 2: Estructuración
  Supervisor -- "Delegación Fase 2" --> Estructurador_Agent
  Estructurador_Agent -- "Planificación de Matrices" --> Estructurador_Exec
```

```
Estructurador_Exec --"Actualización de Estado"--> Supervisor
```

```
%% Fase de Autorización
```

```
Supervisor --"Solicitud de Permiso Computacional"--> Human_Review
```

```
Human_Review --"Rechazo de Parámetros"--> Redactor
```

```
Human_Review --"Aprobación de Parámetros"--> Modelador_Agent
```

```
%% Bucle Operativo 3: Computación
```

```
Modelador_Agent --"Asignación de Hiperparámetros"--> Modelador_Exec
```

```
Modelador_Exec --"Exportación de Resultados Empíricos"--> Supervisor
```

```
%% Bucle Operativo 4: Validación y Cierre
```

```
Supervisor --"Delegación Fase 3"--> Evaluador
```

```
Evaluador --"Aprobación de Métricas"--> Redactor
```

```
Evaluador --"Rechazo y Retroalimentación"--> Supervisor
```

```
%% Terminación
```

```
Redactor --> END
```

La operación del sistema transcurre mediante un mecanismo iterativo, donde la mutación del estado determina la siguiente transición.¹ Al concluir sus tareas respectivas, los nodos subordinados restituyen el control al Supervisor a través de una actualización del historial de transacciones.¹ El elemento crucial en este esquema visual es la barrera impuesta por el nodo de punto de control (Human_Review), que ilustra el paradigma de ejecución duradera. El proceso se detiene asíncronamente y entra en latencia persistiendo los metadatos en disco, salvaguardando de esta manera que procesos computacionales financieramente costosos o algorítmicamente pesados no se inicien bajo configuraciones erróneas sin la ratificación explícita de un técnico especializado.²

5. Hoja de Ruta Fase I: Ingeniería de Prompts Orientada a Codex y LLMs Avanzados

Para que el orquestador interactúe fluidamente con la lógica subyacente, la comunicación humana debe ser reemplazada por una ingeniería de instrucciones (*prompt engineering*) estructurada orientada a sistemas. Cuando se trabaja con modelos cognitivos como OpenAI Codex o GPT-4o para el desarrollo y procesamiento automatizado, el diseño del *prompt* trasciende la formulación de consultas para convertirse en la codificación de directivas deterministas que configuran protocolos de salida estrictos.¹⁹

5.1. Producción Determinista de Salidas Estructuradas JSON

El desafío fundamental al interconectar inteligencia artificial generativa con ejecutores en Python es asegurar que el texto producido pueda ser deserializado mediante funciones

convencionales como `json.loads()`). Un modelo no configurado correctamente tenderá a añadir preámbulos discursivos o confirmaciones conversacionales que corrompen el motor de análisis sintáctico.²¹ La hoja de ruta impone el uso innegociable de la funcionalidad de salidas estructuradas (*Structured Outputs*) habilitada a través del parámetro `response_format: {type: "json_schema"}` soportado por los modelos contemporáneos.²¹

Formato de Salida	Fidelidad en Datos Complejos	Eficiencia Token/Rendimiento	Consideraciones y Casos de Uso en el Proyecto
JSON Estricto	Extremadamente Alta	Costo moderado debido a llaves y comillas	Formato estándar para las interacciones del Agente Limpiador y Estructurador debido a la complejidad de la anidación de diccionarios. ²³
YAML	Alta	Alta eficiencia de tokens y rapidez	Ocasionalmente útil para archivos de configuración estáticos donde se prioriza la legibilidad durante auditorías humanas. ²³
Híbrido CSV / Texto Plano	Baja para esquemas profundos	Altamente eficiente y de generación rápida	Utilizado exclusivamente para tareas de extracción bidimensional simple o listados sin jerarquía. ²³

El sistema incorpora bibliotecas como Pydantic para definir esquemas robustos en Python que obliguen al modelo a apegarse a un molde preestablecido de llaves y valores, previniendo divergencias estructurales entre las peticiones y las respuestas.²²

5.2. Diseño de Plantillas de Restricción para la Lógica Operativa

La programación de los *prompts* debe adherirse a principios arquitectónicos que dividan las instrucciones en bloques fundamentales: Objetivo, Contexto, Restricciones y Condición de Finalización.²⁵ Para los componentes de procesamiento de datos, las directivas incorporan además secuencias de terminación (*Stop Sequences*). El uso de caracteres de escape como } informa al motor de decodificación que debe truncar inmediatamente la generación, evitando fugas de contenido irrelevante.²¹

A continuación, se detalla la semántica estructural que gobernará el *prompt* del Agente Limpiador en la fase de producción:

Rol y Sistema

Actúas como un microservicio backend especializado en normalización y limpieza de calidad de datos corporativos.²¹ Tu respuesta debe ser EXCLUSIVAMENTE un bloque de texto que contenga un objeto JSON válido. No introduzcas encabezados, comentarios explicativos ni disculpas.²¹

Tarea de Análisis

Evalúa las características de la muestra de metadatos proporcionada y resuelve las anomalías aplicando lógica deductiva.

Reglas y Restricciones Estrictas

1. Valores Faltantes: Analiza el conteo de nulidad. Si es estadísticamente insignificante (< 5%), define la imputación de la media. Si corrompe la varianza (> 50%), define el descarte del atributo.³⁰
2. Nomenclatura: Obliga a que todas las columnas mantengan un formato snake_case coherente.³¹
3. Valores Nulos de Sistema: Si un hiperparámetro específico no resulta aplicable al tipo de dato en cuestión, su valor en el diccionario debe ser explícitamente null en lugar de una omisión.²¹

Formato de Salida

Tu generación debe detenerse inmediatamente después de cerrar la estructura JSON principal con el carácter }.²⁷

Si por cualquier eventualidad estadística el agente emite un JSON malformado que arroja una excepción JSONDecodeError durante el análisis léxico de Python, el sistema orquestador está

diseñado para emplear rutinas de recuperación (*self-healing*).²⁸ Este proceso captura la traza del error de compilación y emite una nueva llamada automática al modelo requiriendo la corrección exclusiva del segmento sintáctico corrompido, garantizando que el flujo del sistema no sufra detenciones irreparables.²⁸

6. Hoja de Ruta Fase II: Integración Computacional Distribuida mediante SLURM

La ejecución de este Trabajo de Fin de Máster demanda una potencia de cálculo que sobrepasa sistemáticamente la memoria y los ciclos de procesamiento de una estación de trabajo local, especialmente durante la actividad intensiva del nodo Agente Modelador que entrena arquitecturas neuronales.³³ Consecuentemente, el entorno de despliegue exige la integración con un clúster de supercomputación regido por el gestor de colas y recursos SLURM (*Simple Linux Utility for Resource Management*).³³

6.1. Orquestación Dinámica de Trabajos con Python Subprocess

En un entorno HPC convencional, los usuarios escriben manualmente scripts de procesamiento por lotes para someter sus tareas. Sin embargo, en esta arquitectura, es el agente orquestador quien debe comunicarse con el clúster. El pipeline no se invoca monolíticamente en el nodo de acceso (*login node*); el agente, escrito en Python, aprovecha el módulo subprocess para ensamblar comandos sbatch dinámicos y remitir secuencias de ejecución al planificador.³⁴

Cuando el nodo Modelador establece qué algoritmos híbridos desplegar, el sistema genera programáticamente un archivo de texto transitorio, típicamente denominado slurm.sh.³³ Este documento compila las directrices de paralelización y los requerimientos de hardware exactos.

Directiva SLURM	Función y Racionalidad Arquitectónica en el Proyecto
#SBATCH --partition=gpu	Dirige la tarea a la partición de servidores físicos provistos de aceleración gráfica, indispensable para bibliotecas como PyTorch. ³⁶
#SBATCH --cpus-per-task=4	Asigna múltiples núcleos lógicos para la tarea, permitiendo la vectorización y lectura paralela de <i>DataLoaders</i> en Python. ³⁷
#SBATCH --mem=16G	Reserva estrictamente 16 gigabytes de memoria volátil, evitando que el trabajo sea sacrificado por mecanismos de gestión por

	desbordamiento (OOM). ³⁶
#SBATCH --gres=gpu:1	Solicita explícitamente el uso exclusivo de un dispositivo GPU genérico en el nodo computacional asignado. ³⁶
#SBATCH --time=02:00:00	Fija un límite de pared (<i>wall-clock time</i>) de dos horas. Un tiempo razonable asumiendo el margen adicional del 20% recomendado, evitando monopolizar recursos indefinidamente. ³⁶

Tras inyectar estas definiciones, el script incorpora rutinas para purgar módulos en conflicto (module purge), activar el ecosistema necesario (module load cuda/11.7, conda activate tfm_env) y finalmente despachar la tarea subyacente de Python.³⁴

6.2. Gestión Avanzada de Dependencias y Tareas Paralelas

Los flujos de aprendizaje automático exigen un grado extremo de sincronización. Un proceso de evaluación retrospectiva no puede instanciarse matemáticamente si el entrenamiento previo no ha convergido. Para solucionar esta limitación temporal en entornos asíncronos distribuidos, el orquestador implementa una técnica avanzada apoyándose en las banderas `--parsable` y `--dependency` del comando `sbatch`.⁴⁰

La ejecución del comando con `--parsable` induce a SLURM a silenciar la verbosidad de su salida estándar y retornar exclusivamente el identificador numérico de la tarea asignada (*Job ID*).⁴¹ El entorno Python intercepta este identificador y lo almacena en memoria temporal. De este modo, cuando el Supervisor ordena el envío del subsecuente trabajo de auditoría, adjunta la instrucción condicional `afterok:`.

Bash

```
# Envío automatizado del script de modelado desde el Agente (Simulación)
JOB_ID=$(sbatch --parsable slurm_entrenamiento.sh)
```

```
# El script de evaluación se mantiene latente hasta la finalización íntegra del predecesor
sbatch --dependency=afterok:$JOB_ID slurm_evaluacion.sh
```

La condición `afterok` obliga al demonio de programación de SLURM a retener el trabajo secundario en un estado de espera.⁴¹ Solo si el script de modelado concluye entregando un código de salida (exit code) igual a cero —indicando éxito absoluto— se liberarán los recursos

para el trabajo evaluativo.⁴¹ Si ocurre un fallo catastrófico de convergencia o una excepción de memoria, las tareas pendientes son canceladas en cascada y la traza del desastre se retroalimenta al LangGraph.⁴¹

Asimismo, para contingencias donde el nodo Estructurador decida efectuar una exploración de cuadrícula (*Grid Search*) exhaustiva para múltiples hiperparámetros, el sistema explotará los arreglos de tareas (*Job Arrays*) mediante la etiqueta `--array=1-100`.³⁷ Esto instruye al clúster a multiplicar la instancia, pasando a cada script un valor paramétrico derivado de la variable de entorno `$SLURM_ARRAY_TASK_ID`, facilitando el subprocesamiento en paralelo masivo sin asfixiar la infraestructura de red.⁴⁰

7. Hoja de Ruta Fase III: Transición a Entornos de Producción y Contenerización

Un Trabajo de Fin de Máster de naturaleza meramente experimental a menudo carece de tracción en escenarios empresariales. Para elevar el rigor del proyecto, la arquitectura funcional basada en scripts será abstraída y migrada paulatinamente hacia un producto comercial en modalidad de Software como Servicio (SaaS). El pilar estructural de esta migración requiere el aislamiento lógico e íntegro del sistema mediante tecnologías de contenerización basadas en Docker y orquestación con Docker Compose.⁴⁵

El ensamblaje de la aplicación requerirá la articulación de un ecosistema distribuido donde múltiples microservicios operan coordinadamente. Este diseño no monolítico asegura portabilidad y compatibilidad de dependencias sin importar la infraestructura subyacente, consolidando una arquitectura basada en contenedores independientes.⁴⁵

Microservicio Docker	Herramienta Subyacente	Función en la Topología de Producción
Capa de Interfaz de Aplicación	FastAPI / Uvicorn	Expone rutas asíncronas para la invocación y transmisión de los grafos del sistema multi-agente. ¹¹
Capa de Memoria Relacional	PostgreSQL (pgvector)	Conserva la inmutabilidad de los <i>checkpoints</i> del grafo, almacena los historiales de los hilos asíncronos y aloja bases de conocimiento vectorizadas. ¹¹

Broker de Mensajería Event-Driven	Redis	Actúa como un intermediario de publicaciones y suscripciones (pub/sub), gestionando las transmisiones continuas hacia múltiples clientes. ⁵
Trabajadores en Segundo Plano	Celery / LangGraph Background	Extrae las invocaciones de LLM prolongadas y las lecturas de documentos del bucle de eventos principal, operando asincrónamente para prevenir bloqueos de la red. ⁵

El manifiesto de configuración principal, definido típicamente en un archivo `docker-compose.yml`, orquestará la construcción de estos servicios.⁵ Dicho manifiesto establecerá volúmenes locales persistentes (evitando que los reinicios esporádicos aniquilen las bases de datos transaccionales), definirá la resolución de nombres entre contenedores a través de una red virtual interna aislada, y limitará la superficie de ataque exportando únicamente el puerto necesario para la API FastAPI hacia el exterior.⁵ Adicionalmente, herramientas de observabilidad de LLMs como Langfuse y métricas de infraestructura vía Prometheus se integrarán nativamente para monitorizar el rendimiento y el consumo económico de la red de agentes en tiempo real.⁴⁸

8. Hoja de Ruta Fase IV: Implementación Backend Asíncrona con FastAPI

La abstracción de LangGraph debe acoplarse con un marco web capaz de sostener requerimientos asíncronos masivos. FastAPI, apoyado sobre Starlette y el servidor Uvicorn, emerge como el estándar de la industria gracias a su rendimiento comparable a ecosistemas en Node.js o Go, sumado a una sinergia natural con la validación de tipos Pydantic de Python.⁴

Esta capa backend convierte los objetos lógicos de LangGraph en contratos explícitos de red. Las peticiones enviadas por los clientes no son procesadas de manera arbitraria; cada entrada se rige bajo esquemas Pydantic que garantizan validación, desinfección y serialización estructural, proporcionando a los agentes vectores de entrada fiables.⁵² Esta metodología asegura que desviaciones menores en la estructura de una petición HTTP no corrompan el flujo del LLM en fases profundas del grafo.⁵⁴

El desarrollo contemplará la exposición de dos rutas fundamentales. La primera, una ruta de inicio que lanza el grafo y asocia el contexto a una sesión criptográfica mediante autenticación

con tokens JWT (JSON Web Tokens), posibilitando aislamientos multitenencia.⁵ La segunda ruta, infinitamente más compleja, se destina a la transmisión continua de información (*streaming*), dado que un sistema generativo jerárquico presenta latencias incompatibles con peticiones web transaccionales estándar.¹¹

9. Hoja de Ruta Fase V: Arquitectura del Cliente Frontend en Flutter y React

El componente conclusivo para asegurar la viabilidad del TFM como una aplicación comercializable consiste en el despliegue de interfaces gráficas. La arquitectura frontend se bifracturará estratégicamente. Para herramientas de monitoreo analítico, administración de métricas y gestión de tableros (*dashboards*), se utilizará React aprovechando la flexibilidad que sus librerías de componentes ofrecen en el ecosistema web.⁴ Alternativamente, el cliente principal, concebido para brindar una experiencia de usuario interactiva y fluida orientada a la consulta, adoptará Flutter, facilitando un despliegue cruzado nativo para sistemas operativos móviles y de escritorio bajo una única base de código Dart.⁵

El paradigma de comunicación cliente-servidor representa el mayor desafío en esta capa. Una interacción REST tradicional, donde el cliente espera sincrónicamente la respuesta del servidor, produciría bloqueos de interfaz durante largos periodos de tiempo mientras los agentes deliberan, evalúan métricas o escriben scripts. La fluidez de la experiencia del usuario final depende críticamente de la implementación de flujos de datos asíncronos mediante Server-Sent Events (SSE) o WebSockets bidireccionales.⁵⁵

En el backend, FastAPI en capsula las respuestas parciales del orquestador LangGraph mediante generadores asíncronos (por ejemplo, iterando sobre `app.astream()`), y despacha instantáneamente fragmentos de estado y razonamiento a través de la red empleando el protocolo de flujos de eventos (*Event-Stream*).⁵² En el lado de la interfaz, el componente de Flutter o React se suscribe de manera reactiva a esta tubería de datos.⁵⁶ A medida que los fragmentos de información (como marcadores de progreso del Agente Estructurador o métricas iniciales del nodo Modelador) se transmiten en tiempo real, el gestor de estado de la aplicación actualiza incrementalmente la interfaz.⁶ Esta aproximación emula magistralmente el dinamismo estocástico de aplicaciones de vanguardia y motores de respuesta conversacionales, proporcionando una inmersión visual continua de cómo la inteligencia artificial avanza, corrige y procesa los resultados matemáticos bajo demanda.⁶

10. Implicaciones Finales y Síntesis del Sistema

El esfuerzo investigativo de este Trabajo de Fin de Máster consolida un avance tecnológico pragmático, superando la fase incipiente de experimentación con modelos de lenguaje. El diseño propuesto orquesta una arquitectura multi-agente determinista sobre el marco de LangGraph, demostrando que al imponer metodologías rígidas de control de estado, validación humana, plantillas JSON prescriptivas y segmentación funcional algorítmica, se

erradican en gran medida la entropía y las alucinaciones inherentes al ecosistema generativo.

La estrategia de proyección mediante la utilización de herramientas computacionales de alto rendimiento como SLURM y el consecuente enrutamiento asíncrono hacia aplicaciones contenerizadas mediante Docker, FastAPI, React y Flutter, no solo salvaguarda la escalabilidad vertical y horizontal del proyecto; proporciona un marco epistemológico completo sobre cómo construir aplicaciones de inteligencia artificial orientadas a la producción industrial corporativa. Este diseño asegura una rastreabilidad integral, facilita un control humano meticuloso sobre operaciones críticas y establece un nuevo precedente para los flujos de trabajo autónomos en el análisis profundo de datos.

Obras citadas

1. LangGraph Multi-Agent Supervisor - LangChain Reference Docs, fecha de acceso: abril 30, 2026, <https://reference.langchain.com/python/langgraph-supervisor>
2. LangGraph overview - Docs by LangChain, fecha de acceso: abril 30, 2026, <https://docs.langchain.com/oss/python/langgraph/overview>
3. hoja_ruta.docx
4. Developing a Single Page App with FastAPI and React | TestDriven.io, fecha de acceso: abril 30, 2026, <https://testdriven.io/blog/fastapi-react/>
5. Flutter Web with FastApi and Mysql to implement ChatGPT | by Manna Yang - Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@mannayang1369/chatgpt-flutter-web-cbd433a6bf30>
6. Building a Perplexity AI Clone: A Deep Dive into Flutter, FastAPI, and LLM Integration | by Nahid Amin | Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@nahidamin266/building-a-perplexity-ai-clone-combining-flutter-fastapi-and-llms-11c83262474e>
7. langchain-ai/langgraph: Build resilient language agents as graphs. - GitHub, fecha de acceso: abril 30, 2026, <https://github.com/langchain-ai/langgraph>
8. Building Sequential AI Workflows with LangChain and LangGraph - Level Up Coding, fecha de acceso: abril 30, 2026, <https://levelup.gitconnected.com/building-sequential-ai-workflows-with-langchain-and-langgraph-4e78ff70fc14>
9. LangGraph State Management Patterns | Claude Code Skill - MCP Market, fecha de acceso: abril 30, 2026, <https://mcpmarket.com/tools/skills/langgraph-state-management>
10. Memory - Docs by LangChain, fecha de acceso: abril 30, 2026, <https://docs.langchain.com/oss/python/langgraph/add-memory>
11. Building Production-Ready AI Agent Services: FastAPI + LangGraph Template Deep Dive, fecha de acceso: abril 30, 2026, <https://ranjankumar.in/building-production-ready-ai-agent-services-fastapi-langgraph-template-deep-dive>
12. Persistence - Docs by LangChain, fecha de acceso: abril 30, 2026, <https://docs.langchain.com/oss/python/langgraph/persistence>

13. LangGraph Persistence Guide: Checkpointers & State (2026) | Fastio, fecha de acceso: abril 30, 2026, <https://fast.io/resources/langgraph-persistence/>
14. Best practice for managing LangGraph Postgres checkpoints for short-term memory in production? : r/LangChain - Reddit, fecha de acceso: abril 30, 2026, https://www.reddit.com/r/LangChain/comments/1qna46j/best_practice_for_managing_langgraph_postgres/
15. Building a Supervisor Multi-Agent System with LangGraph Hierarchical Intelligence in Action | by Mani | Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@mnai0377/building-a-supervisor-multi-agent-system-with-langgraph-hierarchical-intelligence-in-action-3e9765af181c>
16. graph_mermaid | langchain_core - LangChain Reference Docs, fecha de acceso: abril 30, 2026, https://reference.langchain.com/python/langchain-core/runnables/graph_mermaid
17. LangGraph - graphing using mermaid, fecha de acceso: abril 30, 2026, <https://forum.langchain.com/t/langgraph-graphing-using-mermaid/2605>
18. Multi-Agents with Supervisor Pattern | by TeeTracker | Mar, 2026 - Medium, fecha de acceso: abril 30, 2026, <https://teetracker.medium.com/multi-agents-with-supervisor-pattern-4013d2cd8c49>
19. Prompt engineering | OpenAI API, fecha de acceso: abril 30, 2026, <https://developers.openai.com/api/docs/guides/prompt-engineering>
20. Output Formatting Strategies: Getting Exactly What You Want, How You Want It | by Manoj Desai | Medium, fecha de acceso: abril 30, 2026, https://medium.com/@the_manoj_desai/output-formatting-strategies-getting-exactly-what-you-want-how-you-want-it-8cebb61bad2d
21. Getting clean Json Outputs from LLM for automations - Reddit, fecha de acceso: abril 30, 2026, https://www.reddit.com/r/automation/comments/1rh0mtp/getting_clean_json_outputs_from_llm_for/
22. Structured model outputs | OpenAI API, fecha de acceso: abril 30, 2026, <https://developers.openai.com/api/docs/guides/structured-outputs>
23. Enhancing structured data generation with GPT-4o evaluating prompt efficiency across prompt styles - Frontiers, fecha de acceso: abril 30, 2026, <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1558938/full>
24. Enhancing structured data generation with GPT-4o evaluating prompt efficiency across prompt styles - PMC, fecha de acceso: abril 30, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11979239/>
25. Best practices - Codex - OpenAI Developers, fecha de acceso: abril 30, 2026, <https://developers.openai.com/codex/learn/best-practices>
26. What's your favorite AGENTS.md prompts you use across all your projects : r/codex - Reddit, fecha de acceso: abril 30, 2026, https://www.reddit.com/r/codex/comments/1qli5m3/whats_your_favorite_agentsmd_prompts_you_use/

27. Prompt Engineering Part 4: Architecting Reliable Outputs with Optimization Techniques | by Akanksha Sinha | Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@akankshasinha247/prompt-engineering-part-4-architecting-reliable-outputs-with-optimization-techniques-7bfa0b1b98c7>
28. JSON prompting for LLMs - IBM Developer, fecha de acceso: abril 30, 2026, <https://developer.ibm.com/articles/json-prompting-llms/>
29. Sourcing useful ChatGPT coding prompts to feature - Codex - OpenAI Developer Community, fecha de acceso: abril 30, 2026, <https://community.openai.com/t/sourcing-useful-chatgpt-coding-prompts-to-feature/1357452>
30. Prompts for Cleaning & Processing data with AI : r/PromptEngineering - Reddit, fecha de acceso: abril 30, 2026, https://www.reddit.com/r/PromptEngineering/comments/16wgsr7/prompts_for_cleaning_processing_data_with_ai/
31. 10 OpenAI Codex Templates That Code Better Than Me | by Bhagya Rana - Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@bhagyarana80/10-openai-codex-templates-that-code-better-than-me-439db0e819b8>
32. Mastering JSON Prompting for LLMs - MachineLearningMastery.com, fecha de acceso: abril 30, 2026, <https://machinelearningmastery.com/mastering-json-prompting-for-llms/>
33. Slurm quickstart - Documentation - LUMI, fecha de acceso: abril 30, 2026, <https://docs.lumi-supercomputer.eu/runjobs/scheduled-jobs/slurm-quickstart/>
34. Python Simple Job Tutorial - research.stern.nyu.edu, fecha de acceso: abril 30, 2026, <https://research.stern.nyu.edu/tutorials/python-simple-job-tutorial>
35. Using Python subprocess to run SLURM script to submit multiple long jobs to queue and waiting for jobs to finish before continuing python script - Stack Overflow, fecha de acceso: abril 30, 2026, <https://stackoverflow.com/questions/43669432/using-python-subprocess-to-run-slurm-script-to-submit-multiple-long-jobs-to-queue>
36. This template is to train a deep learning project by SLURM for allocating single GPU and single server environment. - GitHub, fecha de acceso: abril 30, 2026, <https://github.com/aatizghimire/deep-learning-gpu-slurm-template>
37. Short Tutorial: How to Create a Slurm Script - ARCH Advanced Research Computing, fecha de acceso: abril 30, 2026, <https://www.arch.jhu.edu/short-tutorial-how-to-create-a-slurm-script/>
38. Slurm - Princeton Research Computing, fecha de acceso: abril 30, 2026, <https://researchcomputing.princeton.edu/support/knowledge-base/slurm>
39. Running parallel multiprocessing python scripts with slurm - Stack Overflow, fecha de acceso: abril 30, 2026, <https://stackoverflow.com/questions/77361253/running-parallel-multiprocessing-python-scripts-with-slurm>
40. SLURM Job Dependencies Example - UArizona HPC Documentation, fecha de acceso: abril 30, 2026, https://hpcdocs.hpc.arizona.edu/running_jobs/batch_jobs/job_dependencies/

41. Building pipelines using slurm dependencies - HPC @ NIH, fecha de acceso: abril 30, 2026, https://hpc.nih.gov/docs/job_dependencies.html
42. Best practices for SLURM job pipeline with wrapper scripts - avoiding complex job ID extraction - Stack Overflow, fecha de acceso: abril 30, 2026, <https://stackoverflow.com/questions/79773042/best-practices-for-slurm-job-pipeline-with-wrapper-scripts-avoiding-complex-job-id-extraction>
43. Jobs with Dependencies - Yale Center for Research Computing, fecha de acceso: abril 30, 2026, <https://docs.ycrc.yale.edu/clusters-at-yale/job-scheduling/dependency/>
44. Parallelizing Python scripts with Slurm arrays - Maximilian Pierzyna, fecha de acceso: abril 30, 2026, https://mpier.eu/blog/slurm_python_array/
45. End-to-end testing and deployment of a multi-agent AI system with Docker, LangGraph, and CircleCI, fecha de acceso: abril 30, 2026, <https://circleci.com/blog/end-to-end-testing-and-deployment-of-a-multi-agent-ai-system/>
46. filipkny/langgraph-deploy-demo: This repository will contain an example of how to run a LangGraph with memory using PostgreSQL - GitHub, fecha de acceso: abril 30, 2026, <https://github.com/filipkny/langgraph-deploy-demo>
47. Building an Adaptive RAG System with FastAPI, Langgraph, ChromaDB, PostgreSQL and Docker | by Binuka Jayaweera | Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@binukajayaweera/building-an-adaptive-rag-system-with-fastapi-langgraph-chromadb-postgresql-and-docker-73b49fc736ed>
48. wassim249/fastapi-langgraph-agent-production-ready-template - GitHub, fecha de acceso: abril 30, 2026, <https://github.com/wassim249/fastapi-langgraph-agent-production-ready-template>
49. Self-host standalone servers - Docs by LangChain, fecha de acceso: abril 30, 2026, <https://docs.langchain.com/langsmith/deploy-standalone-server>
50. A production-ready FastAPI template for building AI agent applications with LangGraph ... - GitHub, fecha de acceso: abril 30, 2026, <https://github.com/fangingsong/fastapi-langgraph>
51. Build a FastAPI Backend for Mobile Apps | by Hash Block - Medium, fecha de acceso: abril 30, 2026, <https://medium.com/@connect.hashblock/build-a-fastapi-backend-for-mobile-apps-35cfb9ea82fb>
52. Deployment - LangGraph - Mintlify, fecha de acceso: abril 30, 2026, <https://mintlify.com/langchain-ai/langgraph/guides/deployment>
53. How to use LangGraph within a FastAPI Backend - DEV Community, fecha de acceso: abril 30, 2026, <https://dev.to/anuragkanojiya/how-to-use-langgraph-within-a-fastapi-backend-a-mm>
54. Building Production-Ready AI Backends with FastAPI - DEV Community, fecha de acceso: abril 30, 2026, <https://dev.to/hamluk/building-production-ready-ai-backends-with-fastapi-4352>

55. Building a Real-Time Flutter + FastAPI + Go Application: What Worked, What Didn't, fecha de acceso: abril 30, 2026,
<https://sharjeel-482.medium.com/building-a-real-time-flutter-fastapi-go-application-what-worked-what-didnt-934a579e92d3>
56. Streaming AI Agent with FastAPI & LangGraph (2025-26 Guide) - DEV Community, fecha de acceso: abril 30, 2026,
https://dev.to/kasi_viswanath/streaming-ai-agent-with-fastapi-langgraph-2025-26-guide-1nkn
57. Building a Streaming AI Answer Engine: A Full-Stack Guide with Flutter and FastAPI | by Moyeen Haider | Medium, fecha de acceso: abril 30, 2026,
https://medium.com/@moyeen_haider/building-a-streaming-ai-answer-engine-a-full-stack-guide-with-flutter-and-fastapi-ce2665635285
58. Build a Perplexity Clone: Flutter RAG App using FastAPI and Gemini AI | Machine Learning, fecha de acceso: abril 30, 2026,
<https://www.youtube.com/watch?v=vPbNnHEjnFU>